

Peak Detection and Fitting Code Snippets

Sources started from `resources.txt`:

- SciPy: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.find_peaks.html
- Astropy modeling:
<https://docs.astropy.org/en/latest/api/astropy.modeling.fitting.LevMarLSQFitter.html>
- Astropy Gaussian2D:
https://docs.astropy.org/en/stable/api/astropy.modeling.functional_models.Gaussian2D.html
- lmfit peak models: https://lmfit.github.io/lmfit-py/builtin_models.html
- py-findpeaks overview: <https://github.com/MonsieurV/py-findpeaks>
- mlex_peak_detection: https://github.com/mlexchange/mlex_peak_detection/tree/main
- scikit-image `peak\_local\_max`: <https://scikit-image.org/docs/stable/api/skimage.feature.html>
- pyFAI SAXS/WAXS integration: <https://pyfai.readthedocs.io/en/latest/pyFAI.html>

1D peak detection with SciPy

Use this after reducing SAXS/WAXS/GISAXS/GIWAXS data to a 1D curve, e.g. intensity vs `q`.

```
```python
import numpy as np
from scipy.signal import find_peaks, peak_widths, savgol_filter

def detect_1d_peaks(q, intensity, *, prominence=0.05, min_distance=5):
 y = np.asarray(intensity, dtype=float)
 y = np.nan_to_num(y, nan=np.nanmedian(y))
 y_smooth = savgol_filter(y, window_length=11, polyorder=3)

 baseline = np.percentile(y_smooth, 10)
 y_corrected = y_smooth - baseline

 peaks, props = find_peaks(
 y_corrected,
 prominence=prominence * np.nanmax(y_corrected),
 distance=min_distance,
)
 widths = peak_widths(y_corrected, peaks, rel_height=0.5)

 return [
 {
 "q": float(q[idx]),
 "intensity": float(y_corrected[idx]),
 "prominence": float(props["prominences"][i]),
 "fwhm_samples": float(widths[0][i]),
 }
 for i, idx in enumerate(peaks)
]
...

```

## ## Multi-peak Gaussian fit with lmfit

Use SciPy peak locations as initial guesses, then refine center, width, and amplitude.

```
```python
import numpy as np
from lmfit.models import GaussianModel, LinearModel

```

```

def fit_gaussian_peaks(x, y, peak_indices):
    model = LinearModel(prefix="bg_")
    params = model.make_params(slope=0.0, intercept=float(np.nanmedian(y)))

    for n, idx in enumerate(peak_indices):
        prefix = f"p{n}_"
        peak = GaussianModel(prefix=prefix)
        model += peak
        params.update(
            peak.make_params(
                center=float(x[idx]),
                amplitude=float(max(y[idx], 1e-12)),
                sigma=max(float(np.diff(x).mean() * 3), 1e-12),
            )
        )
        params[f"{prefix}sigma"].min = 0

    result = model.fit(y, params, x=x, nan_policy="omit")
    return result

```

2D local maxima on detector images with scikit-image

Use for candidate spots/rings before geometry conversion or local peak fitting.

```

```python
import numpy as np
from skimage.feature import peak_local_max
from scipy.ndimage import gaussian_filter

```

```

def detect_2d_spots(image, *, min_distance=8, threshold_rel=0.05):
 img = np.asarray(image, dtype=float)
 img = np.nan_to_num(img, nan=np.nanmedian(img))
 img = gaussian_filter(img, sigma=1.0)

 coords = peak_local_max(
 img,
 min_distance=min_distance,
 threshold_abs=threshold_rel * np.nanmax(img),
 exclude_border=False,
)
 return [{"row": int(r), "col": int(c), "intensity": float(img[r, c])} for r, c in
coords]

```

## ## 2D Gaussian refinement with Astropy

Use after candidate spot detection to refine peak center and spread in a small image crop.

```

```python
import numpy as np
from astropy.modeling import fitting, models

```

```

def fit_2d_gaussian_patch(patch):

```

```

patch = np.asarray(patch, dtype=float)
yy, xx = np.mgrid[: patch.shape[0], : patch.shape[1]]

y0, x0 = np.unravel_index(np.nanargmax(patch), patch.shape)
initial = models.Gaussian2D(
    amplitude=float(np.nanmax(patch)),
    x_mean=float(x0),
    y_mean=float(y0),
    x_stddev=2.0,
    y_stddev=2.0,
)

fitter = fitting.LevMarLSQFitter()
fitted = fitter(initial, xx, yy, patch)
return {
    "x": float(fitted.x_mean.value),
    "y": float(fitted.y_mean.value),
    "x_stddev": float(fitted.x_stddev.value),
    "y_stddev": float(fitted.y_stddev.value),
    "amplitude": float(fitted.amplitude.value),
}
...

```

Azimuthal integration with pyFAI, then peak detection

Use for SAXS/WAXS detector images once geometry is known from a `.poni` file.

```

```python
import pyFAI

def integrate_then_find_peaks(image, poni_file, *, npt=1000):
 ai = pyFAI.load(poni_file)
 result = ai.integrate1d(image, npt=npt, unit="q_A^-1")

 q = result.radial
 intensity = result.intensity
 peaks = detect_1d_peaks(q, intensity, prominence=0.03, min_distance=8)
 return q, intensity, peaks
...

```

## ## Simple pipeline shape

```

```python
def analyze_scattering_image(image, poni_file=None):
    if poni_file:
        q, intensity, peaks = integrate_then_find_peaks(image, poni_file)
        return {"mode": "integrated_1d", "q": q, "intensity": intensity, "peaks": peaks}

    spots = detect_2d_spots(image)
    return {"mode": "raw_2d", "spots": spots}
...

```